

Assessments

SE - Introduction to Programming

[A] Section A — Concept Application

SCENARIO: 1 You are a junior developer at a fintech startup building a digital wallet app. The team is debating between Waterfall and Agile methodology. The project has frequently changing requirements and the client expects to review working features every two weeks.

Question: Which development methodology would you recommend for this project, and why? Identify two specific risks the team would face if they chose the wrong methodology for this context.

A.1 I would recommend the Agile methodology for the digital wallet project because the requirements change frequently and the client wants to review working features every two weeks.

Agile divides the project into small development cycles called sprints. At the end of each sprint, the team can demonstrate working features, collect feedback and make improvements.

Two risks of choosing the wrong methodology:

1. Delayed delivery: If the team uses Waterfall, changing requirements may require revisiting completed stages, which can delay the project.
2. Client dissatisfaction: If the team does not regularly review working features, the final application may not meet the client's expectations.

Assessments

SCENARIO: 2 You are collaborating on a college project website with three teammates. Just before a client demo, one teammate pushed incomplete code directly to the main branch, breaking the layout for everyone else on the team.

Question: Describe the Git workflow your team should adopt to prevent this from happening again. Name at least two Git commands your team should use as part of this workflow and explain the role of each command.

A.2 Our team should use a feature-branch workflow instead of allowing everyone to push code directly to the main branch.

Workflow:

1. Each teammate creates a separate branch for their task.
2. They write and test their code on that branch.
3. They commit their changes and push the branch to the remote repository.
4. They create a pull request for code review.
5. After review and successful testing, the code is merged into the main branch.

Command	Purpose
git branch feature-name	Creates a new branch for a feature.
git checkout feature-name	Switches to the feature branch.
git add .	Stages changed files.
git commit -m "message"	Saves changes in a commit.

git push origin feature-name	Uploads the branch to the remote repository.
------------------------------	--

Assessments

git pull	Downloads and integrates remote changes.
----------	--

This workflow helps prevent incomplete code from breaking the main project and gives teammates a chance to review changes before merging.

SCENARIO: 3 You are building an online internship application form for a college placement portal. The form must collect: applicant name, email address, phone number, preferred domain (one of: Web, Data, Mobile, AI), and a resume file upload. The institution requires that no field be left empty and that the email field only accepts valid email formats — without writing any JavaScript.

Question: List the appropriate HTML input types you would use for each of the five fields. Explain how you would use HTML5 built-in validation attributes to enforce the two constraints mentioned, and why these attributes alone are not sufficient for production-level validation.

A.3 The internship application form needs five fields.

Field	HTML input type
Applicant name	text
Email address	email
Phone number	tel
Preferred domain	select dropdown
Resume upload	file

HTML5 validation attributes:

- **required:** Makes a field compulsory.
- **type="email":** Checks whether the entered email has a browser-recognized email format.
- **accept=".pdf,.doc,.docx":** Restricts the file picker to the specified resume formats.

Assessments

```
<form>
  <input type="text" name="name" required>

  <input type="email" name="email" required>

  <input type="tel" name="phone" required>

  <select name="domain" required>
    <option value="">Select domain</option>
    <option>Web</option>
    <option>Data</option>
    <option>Mobile</option>
    <option>AI</option>
  </select>

  <input type="file" name="resume"
    accept=".pdf,.doc,.docx" required>

  <button type="submit">Submit</button>
</form>
```

Why this is not enough for production:

HTML validation can be bypassed by users or automated requests. The server must validate the submitted fields again, check the actual uploaded file type and size, and handle invalid or malicious data securely. Also, type="email" checks format, not whether an email address actually exists.

Assessments

SCENARIO: 4 You are designing a news portal that displays articles in a 3-column grid on desktop screens and in a single-column layout on mobile devices. Your team wants to use Bootstrap's grid system to achieve this without writing any custom media query CSS.

Question: Explain how Bootstrap's grid class system works to produce this responsive layout. Write the Bootstrap class combination you would apply to the article column div, and explain what each class in the combination controls.

A.4 Bootstrap uses a 12-column grid system. We can divide a desktop row into three equal columns by giving each article column four of the twelve grid units.

The class combination is:

```
<div class="col-12 col-md-4">  
  Article content  
</div>
```

Explanation:

- col-12: Uses all 12 columns on extra-small screens, so the article takes the full width.
- col-md-4: From the medium breakpoint upward, each article uses 4 of 12 columns, allowing three articles per row.

The articles should be placed inside a .row, usually inside a .container.

```
<div class="container">  
  <div class="row">  
    <div class="col-12 col-md-4">Article 1</div>  
    <div class="col-12 col-md-4">Article 2</div>  
    <div class="col-12 col-md-4">Article 3</div>  
  </div>  
</div>
```

Assessments

This gives a single-column layout on small screens and three columns on medium and larger screens, without custom media queries.

SCENARIO: 5 You are writing a C program to record daily rainfall data for a month (30 days). You need to store all 30 readings, calculate the monthly average, and then identify which specific days recorded rainfall above that average.

Question: Explain why an array is the correct data structure for this task compared to using 30 separate variables. Describe the two-step logic your program would follow: first to compute the average, then to identify the above-average days — and explain why both steps cannot be done in a single loop.

A.5 An array is suitable because we need to store 30 rainfall readings of the same data type. Instead of creating 30 separate variables, we can store them in one array and access each reading using an index.

For example:

```
float rainfall[30];
```

Two-step logic:

1. Calculate the average: Use a loop to add all 30 readings, then divide the total by 30.
2. Find above-average days: Use another loop to compare each reading with the calculated average and print the day number if its rainfall is greater.

Both steps are separated because we need the final average before we can reliably identify all readings above it. During the first pass, the average is not yet known.

SCENARIO: 6 You are debugging a C program that uses a pointer to traverse a string entered by the user. The program works correctly when the user types a

Assessments

name, but crashes immediately when the input field is left empty (the user just presses Enter).

Question: Explain why the program crashes on an empty string input and describe the check your pointer-based solution must perform before dereferencing the pointer. How does traversing a string using a pointer differ from traversing it using array index notation, and which approach makes the empty-input bug easier to catch?

A.6 In C, an empty string is not an invalid string. It is a valid string containing only the null terminator '\0'.

A pointer-based program may crash if it dereferences a null or uninitialized pointer, or if it moves beyond the valid string and accesses memory incorrectly. An empty string itself should not cause a crash if it is handled correctly.

Before dereferencing, the program should check that the pointer is not NULL. It should also check whether the character it points to is '\0' before attempting to process another character.

Example:

```
if (ptr != NULL && *ptr != '\0') {  
    printf("%c", *ptr); }
```

Difference between the two approaches:

- Array indexing uses expressions like `str[i]`.
- Pointer traversal uses expressions like `*ptr` and advances with `ptr++`.

Array indexing can make it easier for beginners to notice the position being accessed, while pointer traversal requires careful tracking of the pointer's current location. Both approaches need correct empty-string handling and valid bounds.

Assessments

[B] Section B — Practical Coding Tasks

Task 1: Grade Band Checker Build a console program that accepts a student's percentage score and prints their letter grade along with a short motivational message.

Accept a float percentage value as user input using `scanf()`.

Assign a grade using if-else if: A (≥ 90), B (≥ 75), C (≥ 60), D (≥ 45), F (below 45).

Print the assigned grade and a one-line message for each band (e.g., 'B — Good work! Keep pushing.').

Handle invalid input: if the score is outside the range 0–100, print a clear error message and exit the program gracefully.

T.1

```
#include <stdio.h>

int main() {

    float percentage;
    printf("Enter percentage: ");

    if (scanf("%f", &percentage) != 1) {
        printf("Invalid input!\n");
        return 1;
    }

    if (percentage < 0 || percentage > 100) {
```

Assessments

```
printf("Error: Percentage must be between 0 and 100.\n");
return 0;

}

if (percentage >= 90) {
printf("Grade: A\nExcellent work! Keep shining.");
}

else if (percentage >= 75) {
printf("Grade: B\nGood work! Keep pushing.");
}

else if (percentage >= 60) {
printf("Grade: C\nGood effort! You can improve.");
}

else if (percentage >= 45) {
printf("Grade: D\nKeep practicing and improving.");
}

else {
printf("Grade: F\nDon't give up. Try again!"); }
return 0;

}
```

Output example--- > Enter percentage: 82
Grade: B
Good work! Keep pushing.

Task 2: Weekly Study Hours Analyser Build a program that records a student's daily study hours for 7 days, stores them in an array, and generates a performance

Assessments

summary.

Use a for loop to accept 7 float values (one per day) and store them in a float array. Calculate and print the weekly total, daily average, and the day number with the highest study hours.

Print a simple visual bar for each day: the day number followed by one asterisk (*) per hour studied, truncated to the nearest integer (e.g., Day 3: ***).

Reject and re-prompt for any day entry that is negative or greater than 24, before storing it in the array.

T.2

```
#include <stdio.h>
```

```
int main() {
```

```
    float hours[7];
```

```
    float total = 0, average;
```

```
    int highestDay = 0;
```

```
    int i;
```

```
    for (i = 0; i < 7; i++) {
```

```
        do {
```

```
            printf("Enter study hours for Day %d: ", i + 1);
```

```
            if (scanf("%f", &hours[i]) != 1) {
```

```
                printf("Invalid input!\n");
```

```
                return 1; }
    }
```

Assessments

```
        if (hours[i] < 0 || hours[i] > 24) {
            printf("Enter hours between 0 and 24.\n");

        }

    }
    while (hours[i] < 0 || hours[i] >
        total += hours[i];

    if (hours[i] > hours[highestDay]) {

        highestDay = i; }
}

    average = total / 7;
    printf("\nWeekly Total: %.2f hours\n", total);
    printf("Daily Average: %.2f hours\n", average);
    printf("Highest Study Hours: Day %d\n" highestDay + 1);
    printf("\nStudy Chart:\n");

    for (i = 0; i < 7; i++) {

        int j;
        printf("Day %d: ", i + 1);
        for (j = 0; j < (int)hours[i]; j++) {

            printf("*");

        }

        printf("\n");
    }
    return 0; }
```

Task 3: Student Record Manager Build a C program using structures and functions to store, display, and identify the top performer among a group of students.

Assessments

Define a struct Student with fields: name (char[50]), rollno (int), marks (float), and grade (char).

Write a function void assignGrade(struct Student *s) that sets the grade field based on the marks value using the same bands as Task 1.

In main(), accept data for 3 students, call assignGrade() for each, and display all records in a formatted table with column headers.

Write a separate function void printTopper(struct Student[], int n) that prints the name and marks of the student with the highest marks.

T.3 #include <stdio.h>

```
struct Student {  
    char name[50];  
    int rollno;  
    float marks;  
    char grade;  
};
```

```
void assignGrade(struct Student *s) {  
if (s->marks >= 90)  
    s->grade = 'A';  
else if (s->marks >= 75)  
    s->grade = 'B';  
else if (s->marks >= 60)  
    s->grade = 'C';  
else if (s->marks >= 45)  
    s->grade = 'D';  
else s->grade = 'F'; }
```

```
void printTopper(struct Student s[], int n) {  
    int i, top = 0;  
  
    for (i = 1; i < n; i++) {
```

Assessments

```
if (s[i].marks > s[top].marks) {  
    top = i;  
}  
}  
printf("\nTopper: %s\n", s[top].name);  
printf("Marks: %.2f\n", s[top].marks);  
  
}
```

```
int main() {  
    struct Student s[3];  
    int i;  
  
    for (i = 0; i < 3; i++)  
    { printf("\nEnter details of Student %d\n", i + 1);  
      printf("Name: ");  
      scanf(" %49[^\n]", s[i].name);
```

```
      printf("Roll number: ");  
      scanf("%d", &s[i].rollno);
```

```
      printf("Marks: ");  
      scanf("%f", &s[i].marks);
```

```
      assignGrade(&s[i]);  
    }
```

```
    printf("\nSTUDENT RECORDS\n");  
    printf("%-20s %-10s %-10s %-10s\n", "Name", "Roll No", "Marks", "Grade");  
    for (i = 0; i < 3; i++)  
    {  
        printf("%-20s %-10d %-10.2f %-10c\n", s[i].name, s[i].rollno, s[i].marks,  
            s[i].grade);
```

Assessments

```
}  
printTopper(s, 3);  
return 0;  
}
```

Task 4: Personal Expense Logger Build a menu-driven C program that lets a user log and view personal daily expenses, and saves all records to a text file on exit.

Define a struct Expense with fields: category (char[30]) and amount (float).

Allocate an array to store up to 10 Expense entries.

Present a menu with 3 options: (1) Add Expense, (2) View All Expenses, (3) Save & Exit. Loop until the user selects option 3.

In the View option, display all logged expenses in a formatted list and print the running total at the bottom.

On exit (option 3), write all expense records to a file named expenses.txt using fprintf(), with one record per line in the format: category,amount.

T.4

```
#include <stdio.h>  
struct Expense {  
    char category[30];  
  
    float amount;  
};
```

```
int main() {  
    struct Expense e[10];  
    int count = 0;  
    int choice, i;
```

Assessments

```
float total;
do {

    printf("\n--- Expense Logger ---\n");
    printf("1. Add Expense\n");
    printf("2. View All Expenses\n");
    printf("3. Save & Exit\n");
    printf("Enter choice: ");
    scanf("%d", &choice);

    if (choice == 1) {
        if (count >= 10) {

            printf("Expense limit reached!\n");

        } else {

            printf("Enter category: ");
            scanf(" %29[^\n]", e[count].category);

            printf("Enter amount: ");

            scanf("%f", &e[count].amount);
            if (e[count].amount < 0) {
                printf("Invalid amount!\n");
            } else {

                count++;
                printf("Expense added!\n");

            }

        }

    }

}
```

Assessments

```
else if (choice == 2) {

    total = 0;
    printf("\n--- All Expenses ---\n");
    for (i = 0; i < count; i++) {
        printf("%s: Rs. %.2f\n",

            e[i].category, e[i].amount);

        total += e[i].amount;

    }
    printf("Total: Rs. %.2f\n", total);

}
else if (choice == 3) {
    FILE *fp = fopen("expenses.txt", "w");

    if (fp == NULL) {
        printf("Error opening file!\n");
        return 1;

    }

    for (i = 0; i < count; i++) {
        fprintf(fp, "%s,%.2f\n",
            e[i].category, e[i].amount);

    }
    fclose(fp);

    printf("Expenses saved successfully!\n");

}

else {
```

Assessments

```
        printf("Invalid choice!\n");
    }
    } while (choice != 3);
    return 0;
}
```

C Section C — Mini Capstone Project

Mini Project: *Student Productivity Tracker*

Objective: Build a console-based Student Productivity Tracker that logs daily study hours across subjects for a full week, combining arrays, structures, functions, and file handling from Module 3 into a single working program.

Your project must:

The program must be menu-driven with at least 3 options: (1) Log Today's Study Hours, (2) View Weekly Report, (3) Save & Exit.

Define a struct `StudyLog { char subject[40]; float hours[7]; }` and create an array of at least 3 subject records.

Write a function that calculates and displays the weekly total hours and daily average for each subject.

Display a simple text-based progress chart: for each subject, print one filled dot (•) per hour studied that day (truncate to nearest integer).

On exit, save all records to a file named `productivity_log.txt` using `fprintf()`, with each subject's name and 7 daily values written as a single comma-separated line.

Answer

```
#include <stdio.h>
struct StudyLog {
    char subject[40];
    float hours[7];
```

Assessments

```
};

void showReport(struct StudyLog s[], int n) {
    int i, j;

    float total, average;
    printf("\n--- WEEKLY REPORT ---\n");
    for (i = 0; i < n; i++) {

        total = 0;
        for (j = 0; j < 7; j++) {
            total += s[i].hours[j];
        }
        average = total / 7;
        printf("\nSubject: %s\n", s[i].subject);
        printf("Weekly Total: %.2f hours\n", total);
        printf("Daily Average: %.2f hours\n", average);

    }
    printf("\n--- PROGRESS CHART ---\n");
    for (i = 0; i < n; i++) {

        printf("%s: ", s[i].subject);
        for (j = 0; j < 7; j++) {
            int k;

            for (k = 0; k < (int)s[i].hours[j]; k++) {
                printf("*");
            }

            printf(" "); }

        printf("\n"); }
}
```

Assessments

```
}
int main() {

    struct StudyLog s[3] = {

        {"C Programming", {0}},

        {"Mathematics", {0}},

        {"Web Development", {0}}

    };
    int choice, day, i;

    int logged[7] = {0};
    do {
        printf("\n--- Productivity Tracker ---\n");

        printf("1. Log Today's Study Hours\n");
        printf("2. View Weekly Report\n");

        printf("3. Save & Exit\n");

        printf("Enter choice: ");
        scanf("%d", &choice);

        if (choice == 1) {
            printf("Enter day number (1-7): ");

            scanf("%d", &day);
            if (day < 1 || day > 7) {

                printf("Invalid day!\n");
                continue;
            }
        }
    } while (choice != 3);
}
```

Assessments

```
    }
    if (logged[day - 1]) {

        printf("Hours for this day already logged.\n");
        continue;

    }
    for (i = 0; i < 3; i++) {

        do {

            printf("Hours for %s: ", s[i].subject);
            scanf("%f", &s[i].hours[day - 1]);

            if (s[i].hours[day - 1] < 0 ||
                s[i].hours[day - 1] > 24) {

                printf("Enter 0 to 24 hours.\n");
            }

        } while (s[i].hours[day - 1] < 0 ||
                s[i].hours[day - 1] > 24);

    }
    logged[day - 1] = 1;

    printf("Study hours recorded!\n");
}

else if (choice == 2) {
    showReport(s, 3);

}
else if (choice == 3) {
```

Assessments

```
FILE *fp = fopen("productivity_log.txt", "w");
if (fp == NULL) {

    printf("Could not save file!\n");
    return 1;
}
for (i = 0; i < 3; i++) {

    int j;
    fprintf(fp, "%s", s[i].subject);

    for (j = 0; j < 7; j++) {
        fprintf(fp, ",%.2f", s[i].hours[j]);

    }
    fprintf(fp, "\n");

}
fclose(fp);

printf("Productivity data saved!\n");
}

else {
    printf("Invalid choice!\n");

}
} while (choice != 3);

return 0;
}
```

D Section D — AI-Augmented Learning

Assessments

STEP 1 • BUILD WITH AI Use an AI tool of your choice (ChatGPT, Claude, GitHub Copilot, etc.) to help you write a C program that:

Accepts exactly 10 integers from the user using a loop and stores them in an array.
Finds and displays the maximum value, minimum value, and arithmetic mean (displayed as a float with 2 decimal places).

Sorts the array in ascending order using any sorting method and displays the sorted list.

Prints whether the mean is closer to the minimum, closer to the maximum, or exactly midway between them.

STEP 2 • TEST & DEBUG (WITHOUT AI) Then, working without AI, test the code using boundary inputs: (a) all 10 values identical, (b) a mix of positive and negative integers, and (c) a list where the mean falls exactly between min and max. Find at least one bug, edge-case failure, or improvement in the AI's solution and fix it yourself. **SUBMIT** All 3 items required 1 The exact prompt(s) you gave the AI tool. 2 The AI's original code and your corrected version (submit both as separate .c files). 3 A 3–4 line note explaining what you changed and why the AI's original version needed the correction.

Answer

-----step 1-----

```
#include <stdio.h>
int main() {

    int a[10];
    int i, j, temp;
    int min, max, sum = 0;

    float mean;
    printf("Enter 10 integers:\n");
```

Assessments

```
for (i = 0; i < 10; i++) {
scanf("%d", &a[i]);
}
min = max = a[0];
for (i = 0; i < 10; i++) {
sum += a[i];

if (a[i] < min)

    min = a[i];
if (a[i] > max)
max = a[i];
}

mean = sum / 10.0;

for (i = 0; i < 9; i++) {
for (j = 0; j < 9 - i; j++) {
if (a[j] > a[j + 1]) {
temp = a[j];
a[j] = a[j + 1];
a[j + 1] = temp; }
}

}

printf("Minimum: %d\n", min);
printf("Maximum: %d\n", max);
printf("Mean: %.2f\n", mean);
printf("Sorted array: ");

for (i = 0; i < 10; i++)

    printf("%d ", a[i]);
printf("\n");
```

Assessments

```
if (mean - min < max - mean)
printf("Mean is closer to minimum\n");

else if (mean - min > max - mean)
printf("Mean is closer to maximum\n");

else
printf("Mean is exactly midway\n");
return 0;

}
```

-----step 2 -----

One improvement is to check that all ten inputs are valid integers. Another is to use double for the sum and mean, avoiding integer overflow for large inputs and retaining precision.

```
#include <stdio.h>

int main() {
    int a[10];
    int i, j, temp;
    int min, max;
    double sum = 0, mean;
    double dmin, dmax;

    printf("Enter exactly 10 integers:\n");

    for (i = 0; i < 10; i++) {
        printf("Number %d: ", i + 1);

        if (scanf("%d", &a[i]) != 1) {
```

Assessments

```
    printf("Invalid input! Enter integers only.\n");  
    return 1;  
}  
}
```

```
min = max = a[0];
```

```
for (i = 0; i < 10; i++) {  
    sum += a[i];
```

```
    if (a[i] < min)  
        min = a[i];
```

```
    if (a[i] > max)  
        max = a[i];  
}
```

```
mean = sum / 10.0;
```

```
// Bubble sort
```

```
for (i = 0; i < 9; i++) {  
    for (j = 0; j < 9 - i; j++) {  
        if (a[j] > a[j + 1]) {  
            temp = a[j];  
            a[j] = a[j + 1];  
            a[j + 1] = temp;  
        }  
    }  
}
```

```
printf("\nMinimum: %d\n", min);  
printf("Maximum: %d\n", max);  
printf("Mean: %.2f\n", mean);
```

Assessments

```
printf("Sorted array: ");
for (i = 0; i < 10; i++) {
    printf("%d ", a[i]);
}

printf("\n");

dmin = mean - (double)min;
dmax = (double)max - mean;

if (dmin < dmax)
    printf("Mean is closer to minimum.\n");
else if (dmin > dmax)
    printf("Mean is closer to maximum.\n");
else
    printf("Mean is exactly midway.\n");

return 0;
}
```

Test A — All values identical

Input values

5 5 5 5 5 5 5 5 5

Expected result

Minimum = 5, Maximum = 5, Mean = 5.00; exactly midway.

Assessments

Test B — Positive and negative values

Input values

-10 -5 -3 -1 0 1 2 3 5 8

Expected result

Minimum = -10, Maximum = 8, Mean = 0.00; closer to maximum.

Test C — Mean exactly midway

Input values

0 0 0 0 5 5 10 10 10 10

Expected result

Minimum = 0, Maximum = 10, Mean = 5.00; exactly midway.

I improved the original program by checking whether each input is a valid integer before processing it. I also changed the sum and mean variables to double to reduce the risk of overflow and improve precision. I tested the program with identical values, positive and negative values, and a mean exactly halfway between the minimum and maximum. These changes make the program more reliable for the required test cases.

The exact prompt

Write a C program that accepts exactly 10 integers from the user using a loop and stores them in an array. Find and display the maximum value, minimum value and arithmetic mean, with the mean displayed as a float to 2 decimal places. Sort the array in ascending order using any sorting method and display the sorted list. Finally, print whether the mean is closer to the minimum, closer to the maximum, or exactly midway between them. Include comments and use beginner-friendly C programming.

Assessments

Assessments